Semester system security hardening: RFC 9700 and RFC 8555 compliance audit, PKCE mandates, and wildcard TLS configuration

Computer
Semester system product roadmap: phased milestones, security hardening, and multi-tenant custom domain provisioning in a single architecture blueprint
Computer
Draft the custom domain TLS automation workflow
Detail the OIDC identity federation and account linking spec
Show the phased rollout and security hardening roadmap
Sources
93

Semester should implement custom domains, federation, and its app suite on one security spine: a stable OIDC issuer, explicit school-to-account mapping, host-scoped application sessions, verified tenant domains, and automated TLS that never becomes an authorization shortcut.

The immediate direction is: mandate Authorization Code + PKCE for every client, forbid wildcard OAuth redirects, use a verified domain registry for multi-tenant routing, and use platform-managed or ACME-backed certificate automation with explicit activation and rollback gates.

1. Security-hardening blueprint

```
Users / browsers / native apps
            |
            ▼
  Verified hostname + TLS termination
            |
            ▼
  Host registry → candidate tenant context
            |
            ▼
  OIDC authorization server / federation broker
            |
            ▼
  Canonical account + external identity mapping
            |
            ▼
  Membership + capability + policy + consent evaluation
            |
```

```
          ▼
Education apps / Mail / Files / Search / AI
          |
          ▼
Tenant-isolated data, audit, events, reconciliation
```

The host name determines where a request arrives; it does not grant access. Every sensitive request must still resolve the active account, verified membership, role/capability, resource relationship, policy, consent, entitlement, and workflow state.

Semester's existing design instructions already require one canonical account, tenant/membership, policy, consent, audit, and lifecycle system. This must be extended rather than duplicated. [1]

The project architecture snapshot also identifies unresolved tenancy and AI-policy enforcement risks and marks its security/tenancy phase as not ready for activation. That is dated project evidence rather than a live-status claim, but it makes this hardening work a prerequisite for public federation and custom-domain expansion. [2]

2. RFC 9700 compliance audit and PKCE mandate

RFC 9700 is the IETF's OAuth 2.0 Security Best Current Practice. It requires exact redirect URI matching, requires PKCE for public clients, recommends PKCE for confidential clients, says authorization servers must support PKCE, and says the Resource Owner Password Credentials grant must not be used. It also recommends sender-constrained access tokens and requires public-client refresh tokens to be sender-constrained or rotated. [3]

Mandatory Semester OAuth/OIDC policy

Control	Semester requirement	RFC 9700 basis
Grant type	Authorization Code only for user sign-in	Prefer `response_type=code`; avoid tokens in authorization responses
PKCE	Required for every client; accept only `S256`	Public clients must use PKCE; `S256` avoids exposing verifier
Redirect URI	Exact-string match against pre-registered URI	Mandatory except permitted native loopback port handling
Dynamic callbacks	Register each verified custom-domain callback explicitly	Wildcards create code/token theft risk
State and nonce	Per-transaction, cryptographically random, bound to browser/session	CSRF and OIDC transaction defense
Authorization-server mix-up	Bind transaction to expected issuer; validate `iss`	Required when clients support multiple authorization servers
Implicit flow	Disabled	Tokens in authorization responses leak/replay
Password grant	Disabled	RFC 9700 says it must not be used

Control	Semester requirement	RFC 9700 basis
Refresh tokens	Rotation with family reuse detection; revoke family on reuse	Public clients must rotate or sender-constrain
Access tokens	Short-lived, audience-restricted, minimum scopes	Limits blast radius and token misuse
Client authentication	`private_key_jwt` or mutually authenticated TLS for confidential clients where feasible	Asymmetric client authentication recommended
Browser storage	BFF/server-held tokens where possible; no long-lived browser storage	Reduces token exposure
Metadata	Publish issuer metadata and supported PKCE methods	Helps correct, secure configuration
Authorization endpoint	No CORS; browser redirects only	RFC 9700 differentiates endpoint behavior
Open redirects	Prohibited on all callback and continuation routes	Prevents authorization-code and token exfiltration

Required Authorization Code + PKCE flow

```
1. Client creates:
   state, nonce, code_verifier, code_challenge = S256(code_verifier)

2. Client starts authorization:
   response_type=code
   client_id
   exact redirect_uri
   state
   nonce
   code_challenge
   code_challenge_method=S256

3. Semester identity service:
   validates client, exact redirect URI, and S256 method
   stores a short-lived, one-time transaction
   authenticates user or invokes school federation

4. Semester returns:
   authorization code + state
   only to the exact registered redirect URI

5. Client exchanges:
   code + code_verifier

6. Token endpoint:
   verifies code is unused, unexpired, client-bound, redirect-bound,
   and PKCE verifier matches stored challenge

7. Tokens/session:
   issue narrow audience and scopes
   establish host-scoped app session
```

Reject:

- Missing PKCE.
- `plain` PKCE challenges.
- Reused authorization codes.
- A token request with a verifier where the original authorization request had no challenge.
- Redirect URIs that differ by host, path, query normalization, trailing slash, or unexpected port.
- Callbacks to unverified custom domains.
- Authorization requests with an issuer that does not match the transaction-bound school connection.

RFC 9700 says that an authorization server must mitigate PKCE downgrade by only accepting a `code_verifier` when the corresponding authorization request included a `code_challenge`. It also requires authorization servers to support PKCE and recommends advertising supported methods through metadata. [3]

OIDC issuer metadata

Proposed discovery document:

```
{
  "issuer": "https://identity.<semester-domain>",
  "authorization_endpoint": "https://identity.<semester-domain>/authorize",
  "token_endpoint": "https://identity.<semester-domain>/token",
  "jwks_uri": "https://identity.<semester-domain>/jwks.json",
  "response_types_supported": ["code"],
  "grant_types_supported": ["authorization_code", "refresh_token"],
  "code_challenge_methods_supported": ["S256"],
  "token_endpoint_auth_methods_supported": [
    "private_key_jwt",
    "client_secret_basic"
  ]
}
```

This is illustrative. The final document must match the selected maintained OIDC implementation.

Do not use a client secret for native or browser public clients. Do not issue an access token meant for Mail to the academic-records API, even if both services are operated by Semester.

3. OIDC identity federation and account-linking specification

Identity model

```
Semester Account
  account_id: immutable internal identity

External identity
  connection_id + trusted_issuer + subject → account_id

Institution membership
  account_id + tenant_id + lifecycle state

Authorization
  membership + capability + scope + relationship + policy
```

Email addresses and usernames are mutable attributes. They are not authoritative join keys.

Federation connection

Each institution gets an explicitly approved connection record:

```
connection_id
tenant_id
protocol: oidc | saml
trusted_issuer or SAML entity ID
approved metadata/JWKS location
client credentials reference
expected audience/entity ID
claim-mapping version
approved provisioning source
required authentication assurance
lifecycle state
configuration approval and audit trail
```

No user-controlled issuer URLs. No discovery document fetched from an untrusted callback parameter. No automatic mapping solely from `@school.edu`.

Federation flow

```
User selects institution
      ↓
Semester selects active, approved federation connection
      ↓
Transaction binds:
  connection ID
  expected issuer/entity ID
  client ID
  exact callback
  state / nonce / PKCE
      ↓
Redirect to school IdP
      ↓
Validate response against the bound connection
```

```
      ↓
Find external_identity by connection + issuer + subject
      ↓
Resolve or explicitly link Semester Account
      ↓
Resolve membership separately
      ↓
Create session with recorded authentication assurance
```

For multi-provider flows, RFC 9700 requires a mix-up defense; bind the expected issuer to the transaction and validate issuer information in the response.[3]

Account-linking rules

Situation	Required result
Existing external identity mapping	Resolve the mapped Semester Account
New school identity and active Semester Account	Show explicit account-linking approval after both identities are authenticated
New school identity and no active Semester Account	Create/provision only if institution policy allows
Same email on two Semester accounts	Do not merge automatically
Mapping points to another Semester Account	Block reassignment and require verified recovery review
School affiliation ends	Revoke only that membership and its derived authorization
User changes email	Update verified attribute; retain immutable account and identity mappings

A merge, if introduced later, needs a controlled request → identity proof → impact preview → approval → reversible migration window → immutable audit trail. Do not run a background "merge duplicates by email" job.

Security events to audit

- Account creation and external identity linking.
- Federation configuration changes.
- Failed issuer/audience/nonce/signature validation.
- Login from a new connection.
- Session issuance, revocation, and refresh-token reuse detection.
- Membership provisioning, suspension, and offboarding.
- Account-recovery initiation, approval, and completion.
- Domain callback registration and removal.

Never put raw password material, refresh tokens, access tokens, full ID tokens, or protected academic content in generic audit logs.

4. Custom-domain TLS automation workflow

Use two types of custom hostnames:

```
Semester-owned tenant subdomain:
  institution.<semester-domain>

Institution-owned hostname:
  learn.university.example
```

Keep the identity issuer on a stable Semester-controlled host, such as `identity.<semester-domain>`. An institution's branded application hostname should redirect to the central issuer for authentication rather than creating an ungoverned issuer per school.

Domain-binding record

```
domain_binding_id
tenant_id or Semester service ID
normalized_hostname
domain_type: semester_subdomain | institution_custom | mail
verification_state
provider_domain_reference
routing_state
certificate_state
activation_approval
created_by / approved_by
verified_at / last_checked_at
suspended_at / removed_at
```

Constraints:

- One active normalized hostname maps to exactly one service or tenant.
- Normalize lower-case DNS names and internationalized domains consistently.
- Reject IP literals, reserved service hosts, malformed hostnames, and unsupported wildcard requests.
- Retain tombstones for removed hosts to prevent unsafe immediate reassignment.
- Host headers are allowlisted and validated at the edge.

Custom-domain activation workflow

```
1. Request
   Institution admin requests learn.university.example.

2. Validate
   Normalize host; reject collisions, prohibited names, and bad shapes.
   Confirm requester has the institutional authority to make the request.

3. Create pending binding
   State: pending_institution_approval.
```

```
4. Issue ownership proof
   Generate a one-time, expiring DNS TXT challenge.
   Keep the value outside public logs and user-visible URLs.

5. Verify ownership
   Poll authoritative DNS with bounded retries; record verification evidence.
   State: ownership_verified.

6. Bind to delivery provider
   Create/configure domain through the selected hosting provider.
   Do not activate application routing yet.

7. Certificate automation
   Provider/ACME validates the hostname and requests certificate issuance.
   State: certificate_pending → certificate_issued.

8. Route test
   Validate HTTPS hostname/certificate chain, expected edge routing,
   neutral unauthenticated response, and no tenant crossover.

9. Authorize application callback
   Add the exact verified callback URI to approved OAuth client metadata.

10. Activate
    State: active.
    Enable user traffic and start continuous health monitoring.
```

Use explicit approval before step 10. Domain control is not proof that the requested tenant mapping is correct.

Vercel-managed option

If Semester's verified deployment uses Vercel, manage domain bindings with its platform API/SDK and let Vercel issue per-custom-domain certificates. Vercel documents a workflow to add a tenant domain, verify ownership when needed, and automatically issue TLS certificates. [4]

For Semester-owned tenant subdomains such as `institution.<semester-domain>`, Vercel documents a wildcard-domain model: add the apex and wildcard domain, grant the platform DNS-challenge access, and it issues one wildcard certificate for first-level tenant subdomains. A wildcard does not cover deeper levels such as `portal.institution.<semester-domain>`. [4]

ACME-specific controls

RFC 8555 defines ACME: an automated protocol for certificate request, domain-control validation, issuance, renewal, and revocation. ACME clients authenticate requests with signed JWS messages; the protocol uses replay nonces to defend against request replay. [5]

If Semester directly runs or manages ACME automation, require:

- Dedicated ACME account key stored in managed key infrastructure.

- Narrowly scoped DNS credentials limited to the needed `_acme-challenge` records.
- Separate production and non-production ACME accounts.
- Certificate request rate limiting and per-tenant quotas.
- Replay-safe retries using provider/ACME request identifiers.
- CAA-record compatibility checks.
- Certificate Transparency log monitoring.
- Automated renewal with alerting before expiry.
- Documented issuance-failure and emergency certificate-revocation runbook.
- Human approval for suspicious reissuance, hostname reassignment, or broad wildcard changes.

RFC 8555 requires HTTPS for ACME messages, uses signed JWS request bodies, validates domain control through challenges, and provides certificate revocation capabilities. [5]

Wildcard TLS policy

Use case	Recommended certificate model
`tenant.<semester-domain>`	Wildcard certificate for `*.<semester-domain>`
`app.<semester-domain>`	Explicit certificate or covered first-level wildcard
`portal.tenant.<semester-domain>`	Separate hostname/certificate strategy; first-level wildcard does not cover it
`learn.university.example`	Per-domain certificate after ownership verification
Mail transport	Separate SMTP/TLS certificate lifecycle; do not assume web wildcard applies

Avoid wildcard OAuth redirect URIs even where wildcard TLS is correct. TLS issuance proves hostname control, not that arbitrary callback routes should receive authorization codes.

Cookie isolation for tenant subdomains

If untrusted or independently administered tenants can publish content at `tenant.<semester-domain>`, do not use a parent-domain session cookie.

Use host-only cookies such as:

```
Set-Cookie: __Host-semester_session=<opaque>; Secure; HttpOnly; Path=/; SameSite=Lax
```

The `__Host-` prefix requires a host-only cookie with `Secure` and `Path=/`; do not include a `Domain` attribute. Vercel also recommends host-only `__Host-` cookies, CSRF/Origin checks, and Public Suffix List consideration for shared tenant subdomains. [4]

Prefer keeping central authentication and privileged operations on a separate controlled apex from tenant-published content where possible.

Renewal, monitoring, and removal

```
Monitor:
  certificate validity and renewal
  DNS ownership drift
  provider binding status
  HTTPS hostname and chain
  route-to-tenant mapping
  unexpected host traffic
  certificate transparency events

Remove:
  suspend host routing
  disable exact OAuth callback
  invalidate domain routing caches
  remove provider binding
  retain tombstone/audit event
  require new ownership verification before reuse
```

A certificate must never reactivate a removed or reassigned tenant binding.

5. Phased rollout and security hardening roadmap

Phase 1 — Security and identity spine

Primary owners: platform/identity lead, security lead, institutional IT partner.

Workstream	Deliverable	Exit evidence
Canonical identity	Inventory of account IDs, profiles, memberships, auth paths, duplicate mappings	One approved identity source of truth
RFC 9700 hardening	Code-only flow, PKCE S256 enforcement, exact redirects, no password/implicit grants	Automated negative tests for downgrade, replay, redirect, mix-up
Token security	Audience-restricted tokens, rotation, reuse detection, revocation	Token misuse and revocation tests
Federation	One approved school connection with mapping and offboarding	Invalid issuer/audience/nonce/signature tests fail
Tenant authorization	Server-side capability checks plus database/RLS enforcement	Cross-tenant probes fail across API, files, search, AI
Incident readiness	Key rotation, recovery, rollback, monitoring runbooks	Tabletop and operational drills completed

Activation gate: do not enable public custom domains, partner sign-in, or user mailboxes until the identity and tenancy spine passes these tests.

Phase 2 — Custom-domain and mail pilot

Primary owners: platform operations, messaging lead, security and privacy owners.

Workstream	Deliverable	Exit evidence
Domain registry	Approval, verification, routing, certificate, and removal state machine	Collision, takeover, and wrong-tenant probes fail
TLS automation	Managed provider or ACME automation, renewal alerts, rollback workflow	Issue/renew/failure-recovery drill passes
Tenant-host pilot	One institution-approved branded hostname	Host cannot bypass membership checks
Semester Mail pilot	Invitation-only mailboxes, send/receive, delivery/security controls	Unauthorized access and send-as tests fail
Mail operations	SPF/DKIM/DMARC, abuse handling, export/restore, support escalation	Delivery, restore, and abuse drills pass
App integration	User-approved mail-to-task/calendar workflow	Provenance and explicit approval preserved

Activation gate: maintain inbound delivery, export access, and user data integrity even if outbound mail or a custom domain is disabled.

Phase 3 — Multi-school suite and governed ecosystem

Primary owners: product/platform leads, domain owners, developer-platform lead.

Workstream	Deliverable	Exit evidence
Multi-school workspaces	Personal, student, applicant, and alumni contexts	One school cannot query another school's records
Shared services	Calendar, Action Center, Files, scoped Search	Every read/write honors current policy
Offboarding	Graduation, transfer, account/link lifecycle	One membership ends without harming another
Custom-domain scale	Multiple approved domain bindings with lifecycle controls	Renewal, suspension, reassignment tests
Developer pilot	Reviewed OIDC clients, consent, scopes, revocation	Partner cannot exceed authorized audience/scope
Native continuity	Mobile/offline behavior under permission changes	Offline cache cannot retain expired authority

Activation gate: no app, partner, custom hostname, or AI workflow becomes production-enabled without its own audit evidence, rollback plan, support owner, and institution authorization where institutional data is involved.

6. RFC 9700 / RFC 8555 audit checklist

OAuth and OIDC

- [ ] `response_type=code` only for interactive login.
- [ ] PKCE is mandatory; only `S256` accepted.
- [ ] Exact redirect URI comparison; no wildcard callbacks.
- [ ] `state`, nonce, issuer, client, callback, and PKCE are bound to one transaction.
- [ ] Federation callbacks validate the transaction-selected issuer.
- [ ] Implicit and password grants are unavailable.
- [ ] Access tokens are short-lived, narrowly scoped, and audience-restricted.
- [ ] Public-client refresh tokens rotate or are sender-constrained; reuse revokes the family.
- [ ] Confidential clients use strong client authentication where feasible.
- [ ] OIDC signing keys rotate with overlapping JWKS availability.
- [ ] Token, credential, and protected-record values are excluded from logs.
- [ ] Session and authorization cache revocation are tested.

TLS, domains, and ACME

- [ ] Domain binding requires institutional approval and proof of control.
- [ ] DNS verification is separate from provider certificate validation.
- [ ] Hostname routing is allowlisted and fails closed.
- [ ] Certificate state is not equivalent to domain activation.
- [ ] Wildcard certificates are constrained to intended subdomain depth.
- [ ] DNS credentials use least privilege for validation records only.
- [ ] Certificates renew automatically, with expiry and failure alerts.
- [ ] Certificate Transparency monitoring is enabled.
- [ ] Domain removal disables routing and OAuth callbacks before provider cleanup.
- [ ] Host-only, secure, HTTP-only cookies are used for shared-tenant settings.
- [ ] Reassignment requires a fresh verification and approval cycle.

This sequence makes Semester capable of operating a durable education account ecosystem: one stable identity, many bounded institutional relationships, trusted branded domains, and integrated services that do not weaken security or institutional authority.

⁂

1. The Main Semester design system (2) copy/handoff/specifications/role-system-execution-prompt.pdf
2. The Main Semester design system (2) copy/uploads/Semester build Architecture.pdf
3. https://www.rfc-editor.org/rfc/rfc9700.html

4. https://vercel.com/docs/platforms/multi-tenant-platforms/configuring-domains

5. https://www.rfc-editor.org/rfc/rfc8555.html